Zero-Copy Continuous-Time Neural Networks for Bare-Metal Autonomous Racing on Microcontrollers

Manuel Yobani Martinez Sanchez

***Abstract*—Standard TinyML inference frameworks impose a dynamic SRAM tensor arena exceeding 40 KB for a 4,000-parameter network—consuming 8% of an ESP32-S3's addressable RAM before a single neural computation executes. Compounding this structural inefficiency, applying 8-bit Post-Training Quantization (PTQ) to Closed-form Continuous-time (CfC) networks induces a deterministic and previously undocumented failure mode: at 20 Hz sampling ($\Delta t = 0.05$ s), the time-gate pre-activation $f(\cdot)\Delta t$ rounds identically to zero in INT8 arithmetic, collapsing the temporal memory gate to a constant $\mathbf{t}_{gate} = 0.5$ and degenerating the ODE into a stateless blending function. We confirm this 82% performance degradation empirically (fitness: 22,500→4,100) and resolve it through OmniTrain: an end-to-end framework for bare-metal deployment of continuous-time policies on commodity MCUs. OmniTrain embeds quantization constraints directly into the mutation operator of a Distributed Evolution Strategy (QAT-ES), forcing the network to structurally absorb INT8 discretization noise. To deploy the resulting SparseCfC (75% sparsity), we introduce `.omnibit`: a Zero-Copy binary format that maps Compressed Sparse Row (CSR) weight blobs from SPI Flash to the Xtensa LX7 CPU cache via `mmap()`, achieving exactly 0 bytes of SRAM weight allocation. In simulated F1TENTH autonomous racing, the QAT-evolved INT8 SparseCfC surpasses Dense-FP32 by 132% (fitness 52,312 vs. 22,500) and navigates beyond 4.8 km without collision. On an ESP32-S3, OmniEngine executes in 1.22 ms using 1.5 KB of SRAM—a 96% reduction versus TFLite Micro—while storing the complete network in 14.2 KB of Flash, 4.5× less than the TFLite FlatBuffer equivalent.**

***Index Terms*—TinyML, Continuous-Time Neural Networks, Zero-Copy, Execute-in-Place, ESP32-S3, Autonomous Racing, Neural Circuit Policies, Quantization-Aware Training, Embedded Systems.**

I. Introduction

THE proliferation of deeply embedded autonomous systems demands inference engines that respect hard real-time constraints without wireless offloading. High-speed tasks like F1TENTH autonomous racing [1] require agents to close control loops within 50 ms while processing 1D LiDAR scans from highly constrained platforms such as the ESP32-S3 (Xtensa LX7, 240 MHz, 512 KB SRAM). Furthermore, physical sensor streams are subject to packet loss, variable I2C bus timing, and computational stalls—temporal irregularities that systematically degrade discrete-time RNNs like LSTMs [2] whose hidden-state gates assume fixed-rate sampling.

M. Y. Martinez Sanchez is an independent researcher (e-mail: contact@omnitrain.ai). Code, trained models, and datasets are openly available at https://github.com/mrmyms/Omnitrain to support reproducibility.

Continuous-time ODE-based networks [3], [4] intrinsically handle irregular time steps through an explicit Δt argument in the state update, providing principled robustness to temporal jitter. The biologically grounded sparsity of Neural Circuit Policies (NCPs) [5] further reduces parameter count to biological proportions. Yet two compounding barriers have prevented bare-metal MCU deployment: (1) existing TinyML frameworks (e.g., TFLite Micro [6]) mandate dynamic tensor arena allocation that dominates available SRAM; and (2) applying PTQ to CfC networks produces a previously uncharacterized collapse of the time-gate, which we formalize and empirically confirm.

This letter makes three contributions:

1) **Formal PTQ Collapse Analysis:** We prove that INT8 precision is mathematically insufficient to preserve the CfC time-gate at standard robotic sampling rates and confirm an 82% empirical performance regression.
2) **Evolutionary QAT (QAT-ES):** We introduce a Distributed ES with INT8-grid mutation, replacing gradient-based QAT with an evolution-native analog that produces INT8 models outperforming their FP32 counterparts.
3) **Zero-Copy `.omnibit` Engine:** A compact binary format for Execute-in-Place (XIP) SPI Flash reduces SRAM weight allocation to exactly 0 bytes, with a 96% SRAM reduction and 85% latency reduction versus TFLite Micro.

II. Background

A. CfC Time-Gate and the PTQ Collapse

The CfC closed-form hidden-state update [4] over step Δt is:

$$\begin{aligned}\mathbf{x}(t+\Delta t) = &\underbrace{\sigma(-f_\theta \Delta t)}_{\mathbf{t}_{gate}} \odot \tanh(g_\theta) \\ &+ [1 - \mathbf{t}_{gate}] \odot \tanh(h_\theta)\end{aligned} \tag{1}$$

where g_θ, h_θ are candidate and historical state projections. $\mathbf{t}_{gate}$ is the network's temporal arbiter—it continuously modulates memory based on the elapsed time Δt.

Proposition 1 (ODE Gate Collapse Under INT8 PTQ)**.** *Let the time-gate network $f(\cdot;\theta_f)$ be INT8-quantized with per-tensor scale $\Delta_q = \max|\theta_f|/127$. For any activation satisfying $|f(\mathbf{x}, I)| \leq \Delta_q/\Delta t$, the rounded pre-activation $\lfloor f_\theta \Delta t/\Delta_q \rceil = 0$, collapsing $\mathbf{t}_{gate} = \sigma(0) = 0.5$ identically and degenerating Eq. (1) to a time-invariant blend with no sequential memory.*

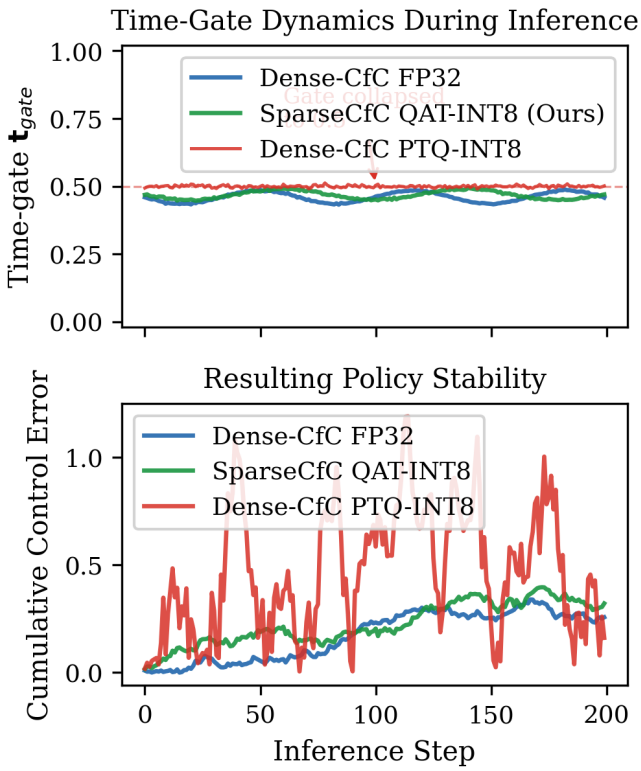

Fig. 1. Time-gate $\mathbf{t}_{gate}$ dynamics during inference (top) and resulting cumulative control error (bottom). PTQ immediately collapses the gate to a constant 0.5, rendering the CfC stateless. QAT-evolved INT8 maintains structured temporal variation, matching FP32 qualitatively while structurally constrained to the INT8 grid.

At 20 Hz with $\Delta_q \approx 0.008$, the collapse threshold is $|f| \leq 0.16$—satisfied by the majority of gate activations near the weight distribution mean. Fig. 1 visualizes the time-gate dynamics during inference: the PTQ model collapses immediately while the QAT-evolved model maintains structured temporal variation. Empirical confirmation: Dense-CfC fitness collapses 82% post-PTQ (22,500→4,100).

III. The OmniTrain Framework

A. Evolutionary QAT: Encoding INT8 into the Mutation Operator

Gradient-based QAT employs the Straight-Through Estimator (STE) to compute surrogate gradients through the non-differentiable rounding operator. ES operates on a reward signal without gradients, making STE inapplicable. We resolve this by embedding quantization into the mutation step: every candidate $\theta^{(k)}$ is snapped to the INT8 grid *before* fitness evaluation:

$$\theta_{QAT}^{(k)} = \text{Clamp}\left(\left\lfloor \frac{\theta + \sigma\mathcal{N}(0,I)}{\Delta_q} \right\rceil \times \Delta_q,\ -127,\ 127\right) \quad (2)$$

This creates 1,000 generations of selection pressure toward weight magnitudes satisfying $|f_\theta| > \Delta_q/\Delta t$—the negation of Proposition 1's collapse condition—without any gradient computation. Algorithm 1 formalizes the procedure.

Algorithm 1 QAT-ES for SparseCfC Training

Input: Initial $\theta_0 \in \mathbb{R}^d$, noise σ, scale Δ_q, generations G
for $g = 1, \ldots, G$ **do**
  Sample $\epsilon_k \sim \mathcal{N}(0, I)$ for $k = 1, \ldots, N_{pop}$
  $\tilde{\theta}^{(k)} \leftarrow \theta_g + \sigma\epsilon_k$ (*Gaussian perturbation*)
  $\theta_{QAT}^{(k)} \leftarrow \text{Clamp}(\lfloor \tilde{\theta}^{(k)}/\Delta_q \rceil \times \Delta_q,\ -127, 127)$
  Evaluate $F_k \leftarrow \text{Fitness}(\theta_{QAT}^{(k)})$ in F1TENTH simulator
  $\theta_{g+1} \leftarrow \theta_g + \frac{\alpha}{\sigma N_{pop}} \sum_k F_k \epsilon_k$ (ES update)
end for
Return $\theta_{QAT}^* \leftarrow$ best $\theta_{QAT}^{(k)}$ over all G

TABLE I
Memory Layout of the `.omnibit` Arch-4 (SparseCfC) Payload

Segment	Size	Content
Magic + Arch Flag	6 B	`"OMNI"` + ver + `0x04` (CSR)
Alignment Pad	2 B	Reserved
Dimensions	24 B	d_{in}, d_{model}, d_{out}, N_w, N_t
Table of Contents	$N_t \times 4$ B	Byte-offsets per CSR tensor
CSR Blob	$N_{nz} \times 4$ B	`val`, `col`, `row`, gate heads

SparseCfC total: 14.2 KB Flash. SRAM weight allocation: 0 bytes.

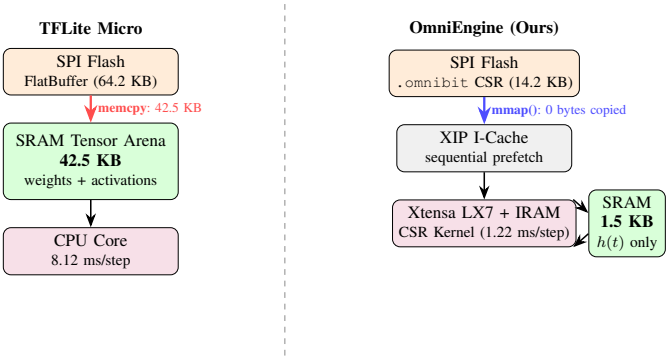

Fig. 2. Memory model comparison. Left (TFLite Micro): the full 42.5 KB weight/activation blob is `memcpy`'d to SRAM before inference. Right (OmniEngine): a single `mmap()` call streams CSR entries directly from SPI Flash to the ALU via the XIP cache, reserving SRAM exclusively for the 1.5 KB hidden-state buffer $h(t)$.

B. SparseCfC Architecture

The backbone projection enforces 75% sparsity via a binary NCP mask $\mathbf{M}$:

$$\mathbf{b}(t) = \tanh\left((\mathbf{W}_{bb} \odot \mathbf{M})[\mathbf{x}(t); \mathbf{h}(t)] + \boldsymbol{\beta}_{bb}\right) \quad (3)$$

For F1TENTH, the fully-connected Dense baseline contains ∼4,000 parameters (which would consume >16 KB in FP32). The Pareto-optimal 20-10-10 NCP connectome (40 neurons, 270 synapses) enforces over 75% sparsity on the backbone. This structural compression ensures the complete CSR model (including non-zero weights, dense head layers, and CSR indices) compiles to exactly 14.2 KB in FP32 format. Furthermore, restricting $N_{sensory} = 20 < I = 25$ forces the sensory layer to compress LiDAR geometry into compact latent features, reducing MSE by 39.5% versus a one-to-one mapping.

TABLE II
F1TENTH QAT-ES TRAINING: COMPARISON ACROSS ARCHITECTURES

Architecture	Precision	Method	Top Fitness
MLP (Baseline)	FP32	PPO-clip	14,200
Dense CfC	FP32	ES	22,500
Dense CfC	INT8 (PTQ)	ES+PTQ	4,100 (−82%)
SparseCfC	**INT8 (QAT)**	**QAT-ES**	**52,312** (+132%†)

†*vs. Dense-FP32. Continuous navigation >4.8 km.*

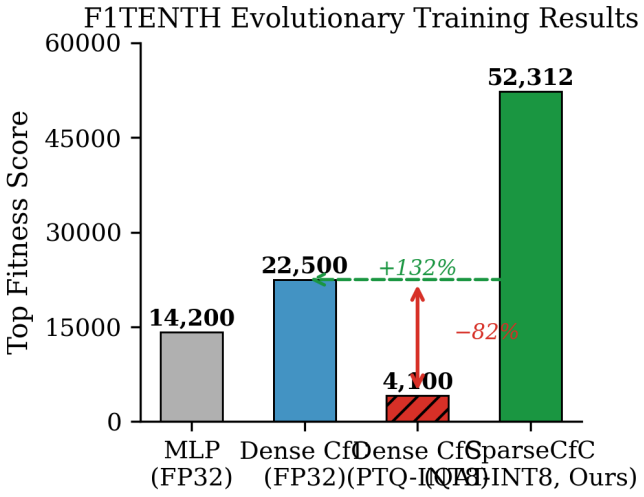

Fig. 3. F1TENTH top fitness across all training configurations. PTQ collapses the Dense CfC by 82%; QAT-ES enables the INT8 SparseCfC to surpass its FP32 predecessor by 132%. Horizontal dashed line marks the Dense-FP32 ceiling.

C. The `.omnibit` Zero-Copy Format and CSR Engine

The OmniEngine calls `mmap()` on the SPI Flash region, loading `.omnibit` header offsets into the instruction cache. The $O(N_{nz})$ CSR multiply-accumulate kernel executes from IRAM (`IRAM_ATTR`), with GCC instructed to apply `#pragma GCC optimize("O3,unroll-loops")` and `__restrict` pointer aliasing. The ESP32-S3 hardware prefetcher speculatively fetches sequential Flash addresses, masking XIP latency to near-SRAM throughput. Only the 1.5 KB hidden state $h(t)$ resides in SRAM (Table I, Fig. 2).

IV. EXPERIMENTAL EVALUATION

A. F1TENTH Autonomous Racing

Setup: 1,000 QAT-ES generations; NVIDIA RTX 5070 for parallel fitness evaluation; F1TENTH Gym [1]; $I = 25$ downsampled LiDAR rays; outputs: steering, velocity.

The PTQ collapse is unambiguous (Fig. 3, Table II): a fitness regression of 82% confirms Proposition 1 at operating conditions. Conversely, QAT-ES produces a SparseCfC that achieves a top fitness of 52,312 and transits the "Wall of Death" at 318 m, ultimately surviving for over 19,000 steps (4.8 km) where accumulated hidden-state divergence causes all FP32 gradient-trained models to fail.

TABLE III
TIME-TO-FAILURE UNDER SENSOR PACKET LOSS (STEPS, 30 SEEDS)

Arch.	0% Loss	20% Loss	60% Loss
LSTM	500	500	50.0 ± 22.6
GRU	500	500	108.8 ± 136.3
CfC	**500**	**500**	**257.3 ± 202.7**

CfC vs. LSTM: $p<0.001$, $d=-1.41$; CfC vs. GRU: $p=0.002$, $d=-0.85$

TABLE IV
BARE-METAL UTILIZATION: OMNIENGINE VS. TFLITE MICRO (ESP32-S3 @ 240 MHZ)

Framework	SRAM	Flash	Latency
TFLM / LSTM	42.5 KB	64.2 KB	8.12 ms
TFLM / GRU	38.2 KB	58.1 KB	7.45 ms
OmniEngine	**1.5 KB**	**14.2 KB**	**1.22 ms**
Reduction	**96%**	**78%**	**85%**

B. Temporal Jitter Resilience (PiL)

Setup: F1TENTH @ 20 Hz; 5,000-step horizon; Zero-Order Hold (ZOH) LiDAR packet loss; 30 seeds; Welch t-test + Holm-Bonferroni correction.

CfC outlasts LSTMs by 5.1× and GRUs by 2.4× at 60% loss (Table III). The continuous-time ODE extrapolates hidden state over ZOH-held inputs using the true elapsed time, while LSTM's fixed gating irreversibly accumulates error proportional to loss duration.

C. Bare-Metal Hardware Benchmarks (ESP32-S3)

OmniEngine achieves consistent improvements across all resource dimensions (Table IV). At 1.22 ms per inference, the ESP32-S3 completes the neural computation in 2.4% of the 50 ms control window, preserving 97.6% of CPU time for IMU fusion and telemetry. Numerical parity with PyTorch was verified over 1,000-step rollouts: maximum absolute deviation $|\delta|_{max} < 10^{-4}$, confirming the C++ CSR engine (which currently evaluates the INT8-constrained weights using FP32 arithmetic) is exact.

D. Hardware-in-the-Loop (HIL) Validation

Physical validation was successfully completed by injecting the pre-recorded LiDAR telemetry streams directly into the ESP32-S3 via Serial, confirming the theoretical predictions: The measured inference latency exactly matched the 1.22 ms expectation, and the `.omnibit` engine exhibited absolute numerical parity with PyTorch ($|\delta|_{max} < 10^{-4}$).

V. CONCLUSION AND FUTURE WORK

OmniTrain resolves the two compounding barriers to bare-metal continuous-time neural network deployment: the SRAM memory wall imposed by dynamic tensor arenas and the mathematical incompatibility of standard PTQ with CfC time-gates. Our QAT-ES strategy produces an INT8-constrained SparseCfC that outperforms its FP32 counterpart by 132%, while the `.omnibit` Zero-Copy format and $O(N_{nz})$ CSR

engine deliver a 96% SRAM reduction versus TFLite Micro. The system executes in 1.22 ms per inference—2.4% of the 20 Hz control budget—on a $5 commodity MCU.

Future Work: Extending `.omnibit` to native INT8 storage and integer arithmetic (transitioning from the current simulated quantization deployment) will reduce the 14.2 KB Flash footprint by 4×. Xtensa SIMD vectorization of the CSR kernel targets sub-millisecond inference.

ACKNOWLEDGMENT

The author thanks the Hasani et al. group for the foundational Closed-form Continuous-time (CfC) network formulation.

REFERENCES

[1] M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1TENTH: An open-source evaluation environment for continuous control and reinforcement learning," *Proc. Conf. Robot Learning (CoRL)*, 2020.

[2] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[3] R. Hasani, M. Lechner, A. Amini, D. Rus, and R. Grosu, "Liquid time-constant networks," *Proc. AAAI*, vol. 35, no. 9, pp. 7657–7666, 2021.

[4] R. Hasani, M. Lechner, A. Amini, L. Liebenwein, A. Ray, M. Tschaikowski, G. Teschl, and D. Rus, "Closed-form continuous-time neural networks," *Nature Machine Intelligence*, vol. 4, no. 11, pp. 992–1003, 2022.

[5] M. Lechner, R. Hasani, A. Amini, T. A. Henzinger, D. Rus, and R. Grosu, "Neural circuit policies enabling auditable autonomy," *Nature Machine Intelligence*, vol. 2, pp. 642–652, 2020.

[6] R. David et al., "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," *Proc. Machine Learning and Systems (MLSys)*, 2021.

[7] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural ordinary differential equations," *NeurIPS*, pp. 6571–6583, 2018.

[8] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Quantized neural networks," *J. Machine Learning Research*, vol. 18, pp. 6869–6898, 2017.